

SHUROQ ENTERPRISE ERP

Mobile Application Launch Plan & Technical Architecture Blueprint

Document: PRD-MOB-2026-01 | Date: 02/09/2026 | Author: PRANESH M S

1. Executive Summary & Business Justification

This document establishes the official engineering proposal and execution plan to launch the Shuroq Enterprise ERP as a native Android application on the Google Play Store for internal company staff. The objective is to provide employees, managers, and executives with secure, instant mobile access to corporate workflows (HRM, attendance, leaves, payroll, project tracking, inventory, and CRM).

Following leadership guidance to prioritize rapid time-to-market and reliable staff onboarding, we evaluated two architectural pathways: a Full Native Rewrite in Flutter vs. a Flutter Native Hybrid Shell (Advanced InAppWebView). Based on technical feasibility, cost efficiency, and 100% feature parity on Day 1, the Flutter Hybrid Shell architecture is the strongly recommended solution.

Strategic Architecture Comparison

EVALUATION PARAMETER	FULL NATIVE REWRITE	FLUTTER HYBRID SHELL (RECOMMENDED)
Time to Market	12 – 16 Weeks (Multi-sprint dev)	5 – 7 Working Days (Immediate pilot)
Day 1 Feature Parity	Partial (1-2 modules initially)	100% Instant Parity (All 11 modules ready)
Maintenance Burden	Dual codebases (Web + Mobile app)	Single codebase (Web updates sync to app)
Feature Release Cycle	Requires Play Store app approval	Zero-downtime Instant OTA updates
Native Capabilities	Full native hardware access	Full native (Push, Biometrics, Files, Camera)
Offline Resilience	Complex local DB sync	Native Network Watcher + Branded Cache
Total Engineering Cost	High (Dedicated mobile squad)	Extremely low (Leverages live platform)

Core Strategic Objectives

- ' Deliver a high-performance Android APK and Google Play Store release for internal staff within 7 business days.
- ' Guarantee 100% feature availability across all 11 enterprise modules without re-engineering complex business logic.
- ' Equip the mobile shell with essential native device integrations: Push Notifications, Biometrics, and File Download Managers.
- ' Ensure full compliance with Google Play Store Developer Guidelines for WebView-powered enterprise applications.

2. Flutter Hybrid Architecture & Core Engine

The proposed solution encapsulates the responsive Shuroq ERP Next.js production web platform (hosted at <https://erp-eight-gules.vercel.app>) within a hardened Flutter Android container. Communication between native hardware layers and the web application is handled via a secure bidirectional JavaScript bridge.

1. Core Engine: flutter_inappwebview v6+

Standard WebViews (such as the default Android WebView or basic webview_flutter plugin) lack support for critical enterprise features like file downloads, scoped storage, camera permissions, and session caching. We will utilize flutter_inappwebview 6.0+, configured with:

Hardware Acceleration
Enables 60fps smooth scrolling, GPU rendering, and fluid gesture handling indistinguishable from native UI.
Cookie & Token Persistence
Retains JWT authentication tokens and HTTP cookies in native encrypted storage so staff remain logged in.
Custom User-Agent
Appends "ShuroqERP-Mobile/1.0" to HTTP headers, allowing backend telemetry to distinguish app vs desktop traffic.
Cache Management
Leverages Web Cache Mode (LOAD_DEFAULT) for instant page transitions while maintaining real-time data freshness.

2. Biometric Security Layer (local_auth)

Because staff mobile devices may be shared or accessed in casual environments, an optional biometric authentication gate will be integrated using Flutter local_auth. When enabled, launching or resuming the ERP app prompts Fingerprint or Face Unlock before revealing active sessions.

3. Network Watcher & Offline Fallback

In traditional WebView wrappers, losing internet displays an unsightly Chromium error page ("ERR_INTERNET_DISCONNECTED"). In our architecture, the connectivity_plus background stream monitors network state continuously. If offline, the app displays a branded, modern offline screen with connection diagnostics and an auto-retry button.

4. App Security & SSL Pinning

To guarantee enterprise data privacy, cleartext HTTP traffic is strictly blocked (android:usesCleartextTraffic="false"). The WebView enforces TLS 1.3 HTTPS validation against all backend API routes and Supabase endpoints.

3. Native Hardware Bridges & Device Features

To satisfy Google Play Store quality standards and provide a premium user experience, the mobile application features four native bridge integrations that bridge web events to Android device APIs.

Bridge 1: Android Hardware Back Navigation (PopScope)

A universal flaw in naive webview apps is that tapping the Android back gesture closes the application. In our Flutter shell, we implement a PopScope history stack handler:

Navigation Logic Stack:

- If Webview `canGoBack() == true` -> Calls `webViewController.goBack()` within the ERP module.
- If at root (Dashboard) -> Triggers native Flutter dialog: "Are you sure you want to exit Shuroq ERP?".

Bridge 2: Native Document & PDF Download Interceptor

Employees regularly download Offer Letters, Payslips, Attendance summaries, and Ledger reports. WebViews cannot directly write to user storage on Android 10+ without scoped storage handlers:

- Download requests are intercepted via `InAppWebView` `onDownloadStart` listener.
- Files are downloaded using native `dio` / `flutter_downloader` directly into the public Android Downloads folder.
- A native status bar notification is displayed upon completion with one-tap opening via `open_filex`.

Bridge 3: Camera & File Upload Integration

Onboarding identity document uploads (PAN cards, Aadhaar scans, resumes) and support ticket attachments require hardware camera and file picker access. The Flutter `AndroidManifest` will declare `CAMERA`, `READ_MEDIA_IMAGES`, and `READ_EXTERNAL_STORAGE` permissions, seamlessly fulfilling HTML file inputs (`<input type="file">`).

Bridge 4: Push Notifications & Deep Linking (Firebase Cloud Messaging)

Firebase Cloud Messaging (FCM) is integrated at the Flutter native layer. When HR approves a leave, a task is assigned, or a company holiday is declared, the backend dispatches a push notification. Tapping the alert deep-links the staff member straight to the relevant module.

4. Detailed 7-Day Implementation Blueprint

The implementation plan is structured into a rigorous 7-day engineering schedule. Each day focuses on specific deliverables with automated verification checkpoints.

DAY 1 Project Setup, Branding & InAppWebView Core

- Initialize production Flutter project with package com.shuroq.erp.
- Configure flutter_inappwebview 6.0+ with hardware acceleration and JavaScript enabled.
- Implement native Android splash screen featuring official Shuroq logo and brand colors.
- Embed live production Next.js frontend (<https://erp-eight-gules.vercel.app>).
- Configure persistent CookieManager and local storage token retention.

Key Deliverable: Functional Android APK loading the live ERP dashboard on real physical devices.

DAY 2 Native Back Navigation, Pull-to-Refresh & Offline Watcher

- Implement PopScope back button navigation hierarchy (intercepts hardware back button).
- Integrate native PullToRefreshController for seamless page reload and data refresh.
- Implement connectivity_plus listener to detect WiFi/Cellular loss.
- Design branded offline fallback screen with "Check Connection & Retry" action.

Key Deliverable: Smooth back-navigation without app exit and graceful offline state handling.

DAY 3 Document Download Interceptor & File Uploads

- Implement onDownloadStart listener to catch all PDF and Excel export triggers.
- Integrate path_provider and open_filex to route downloads to the public Downloads folder.
- Request and handle Android runtime permissions (CAMERA, READ_MEDIA_IMAGES).
- Verify file uploads for employee identity documents (Aadhaar/PAN) and project files.

Key Deliverable: Full document lifecycle working natively: downloading payslips & uploading KYC files.

DAY 4 Firebase Push Notifications & Biometric App Lock

- Create Firebase project and configure google-services.json in android/app.
- Implement firebase_messaging background and foreground push notification handlers.
- Integrate local_auth biometric lock (Fingerprint / Face ID) with fallback PIN.
- Connect FCM device registration token with NestJS backend notification service.

Key Deliverable: Staff receive native notification badges and can lock the app with biometrics.

DAY 5

Comprehensive Cross-Device Testing & Quality Assurance

- Verify all 11 modules on various Android screen resolutions (compact phones, phablets, tablets).
- Verify strict DD/MM/YYYY date formatting across Offer Letters, attendance, and payroll screens.
- Validate Indian Rupee currency displays (₹) and Indian number formatting.
- Stress test token expiration, session renewal, and automatic re-authentication.
- Validate performance under low-bandwidth (2G/3G) simulated network conditions.

Key Deliverable: Zero-crash QA report across real target test devices.

DAY 6

Keystore Signing, ProGuard Hardening & Store Assets

- Generate production cryptographic keystore (shuroq-release-key.jks) with 2048-bit RSA.
- Configure android/key.properties and Gradle release signing configuration.
- Enable R8/ProGuard code obfuscation and resource shrinking to reduce APK footprint.
- Prepare all Google Play Store visual assets:
 - High-res App Icon (512x512 PNG, 32-bit color)
 - Feature Graphic (1024x500 PNG/JPEG)
 - 4-6 Phone Screenshots (1080x2400) showcasing HRM, Projects, and Financial Ledger
- Publish organizational App Privacy Policy on <https://shuroq.com/privacy>.

Key Deliverable: Signed release Android App Bundle (.aab) and complete Play Store media kit.

DAY 7

Google Play Console Release & Internal Staff Rollout

- Upload release App Bundle to Google Play Console Internal Testing track.
- Configure store listing, content rating questionnaire, and target audience declaration.
- Provide Google Play review team with dedicated test credentials (demo employee account).
- Compile standalone release APK (shuroq-erp-v1.0.apk) for direct employee installation.
- Distribute internal download link and guide to company staff for pilot onboarding.

Key Deliverable: Active Google Play Store review track & distributed APK in staff hands.

Build Artifact Outputs

Android App Bundle (.aab): Target format for Google Play Store upload with dynamic feature optimization.

Universal Release APK (.apk): Self-contained 18MB package for direct internal sideloading via staff portal.

Release Keystore (keystore.jks): Securely stored master deployment credential for all future app updates.

5. Google Play Store Compliance & Governance

Google maintains strict policies regarding WebView applications (Google Play Developer Program Policy - Webview spam policy). To guarantee immediate approval without rejections:

- ' **Added Value Requirement:**The app is not a bare wrapper: it embeds native hardware push notifications, biometric security, download file managers, and native camera integration.
- ' **App Privacy Policy**A publicly accessible privacy policy clearly stating employee data collection (name, contact, credentials) for enterprise authentication.
- ' **Reviewer Account Credentials**A restricted test employee login provided in Play Console App Access settings so Google reviewers can inspect the UI behind the login screen.
- ' **Data Safety Declarations**Accurately declaring that data is encrypted in transit via HTTPS and that accounts are managed by the employer with strict RBAC.

6. Action Items & Approvals Required

To commence Day 1 execution immediately, the following company items are requested:

- [1] Approval of this architecture and execution roadmap by Team Lead & Engineering Management.
- [2] Google Play Console Account Access or invitation (\$25 one-time developer registration).
- [3] Confirmation of official App Name: "Shuroq Enterprise ERP".

Prepared By: PRANESH M S Software Engineering / Mobile Lead	Approved By: Team Lead / Engineering Management
--	---

